

Object-Oriented Programming Code Challenge

Freight & Logistics Shipping Cost Management System

An international logistics company is developing a **Freight & Logistics Shipping Cost Management System** to manage customer shipments, calculate transportation costs, estimate delivery expenses, apply discounts, and manage shipment history. As a software engineer, your task is to design a software system using **Object-Oriented Programming (OOP)** in **JavaScript**. This challenge is intended to reinforce all OOP concepts studied in class and requires you to think like a professional software engineer. **Do not use frameworks.**

Concepts to Apply

- 1 Classes and Objects
- 2 Constructors
- 3 Inheritance (extends)
- 4 Encapsulation
- 5 Public Methods
- 6 Private Methods (#)
- 7 Protected Methods (⌋)
- 8 Static Methods
- 9 Arrays of Objects
- 10 Validation Logic
- 11 Business Rules
- 12 State Management
- 13 Mathematical Operations

System Requirements

1. Parent Class: Person

Properties:

- fullName
- customerId

- country

Protected Method (Convention)

Create a protected method called **`_generateShipmentCode()`** that generates a random shipment code using the format: **SHP-5821**. This method should only be used internally or by child classes.

2. Child Class: Customer

The class must inherit from **Person**.

Additional Properties:

- membershipLevel (standard, premium, vip)
- shipments → empty array by default

3. Private Methods

`#validateShipment(shipment)`

Return true only if package name exists, weight > 0, distance > 0, and base shipping price > 0. Otherwise, return false.

`#calculateDiscount(total)`

Apply discounts according to membership level:

- standard → 0%
- premium → 10%
- vip → 20%

4. Public Methods

`createShipment(packageName, weight, distance, basePricePerKm, shippingPriority, insured)`

This method must:

- 1 Validate shipment information.
- 2 Calculate distance cost = distance × basePricePerKm.
- 3 Calculate weight cost = weight × 2.
- 4 Apply shipping priority costs (standard = \$0, express = \$50, urgent = \$100).
- 5 If shipment is insured, calculate insurance as 10% of subtotal.
- 6 Apply membership discount.
- 7 Generate a shipment code.
- 8 Create a shipment object and store it in the shipments array.

9 Return shipment summary.

reportLateDelivery(shipmentCode, daysLate)

Calculate a penalty of **\$25 per late day**. If shipment code does not exist, return an error message.

showShipmentHistory()

Return all shipment history.

5. Static Methods

- `convertKilogramsToPounds(kg)`: convert kilograms to pounds.
- `calculateAverageShippingCost(costs)`: return average shipping cost from an array.

Important Rules

- 1 Do NOT expose private logic outside the class.
- 2 Apply encapsulation correctly.
- 3 Use inheritance properly.
- 4 Keep methods responsible for one task.
- 5 Use at least one array method (reduce, find, findIndex, filter, etc.).
- 6 Test the system using at least two customer objects.
- 7 Follow clean code practices.

Deliverables

- 1 UML Class Diagram
- 2 JavaScript Solution
- 3 Test Cases
- 4 Console Output Examples
- 5 Short explanation of where OOP concepts were applied